

La méthode du recuit simulé n'a pas l'air de fonctionner, qu'en penser ?

June 19, 2026

1 Modifications du calcul du score

En testant l'implémentation du recuit simulé, je me suis aperçu que la valeur du score dépend essentiellement du CE et non du CIP. Une diminution difficile à obtenir est rendu invisible sur le score si le CE est très faiblement augmenté, même d'une augmentation qu'il serait facile d'annuler. Cela dit le score tel qu'il est calculé respecte bien la condition si on diminue CE sans modifier CIP, ou si on diminue CIP sans modifier CE, on augmente bien le score.

À mon avis, Le calcul du score n'est pas la raison pour laquelle l'algorithme du recuit simulé ne fonctionne pas mais je propose quand même d'équilibrer l'impact du CE et du CIP sur le score.

Pour généraliser l'idée, je note $z_0, \dots, z_{I-1}$ les I valeurs que je cherche à minimiser. Autrement dit ici $I = 2$ et z_0 désigne CE et z_1 désigne CIP. Je propose d'utiliser des tirages aléatoires indépendants (c'est-à-dire sans être modifiés par une technique d'optimisation) pour évaluer la moyenne et l'écart-type des valeurs de z_i obtenues, je note ces moyennes et écart-types $\mu_i^{(z)}$ et $\sigma_i^{(z)}$. Je propose alors

$$\text{score} = \frac{1}{I} \sum_{i=0}^{I-1} \frac{\mu_i^{(z)} - z_i}{\sigma_i^{(z)}} \quad (1)$$

Effectivement on n'a plus l'interprétation de z_i comme une probabilité, mais on peut considérer que si les $\frac{1}{I} \frac{\mu_i^{(z)} - z_i}{\sigma_i^{(z)}}$ étaient des logarithmes de probabilités indépendantes alors le score serait le logarithme de la probabilité résultante.

Les $\mu_i^{(z)}$ n'ont pas d'incidences réelles. Les $\sigma_i^{(z)}$ en ont une et leur estimation est en réalité discutable, parce que quand on s'approche de la bonne solution, les valeurs pourraient tout à fait être différentes.

2 Recherche d'une direction privilégiée

En observant l'algorithme du recuit-simulé dans deux configurations, l'une où le score est essentiellement dirigé par la minimisation du CE et l'autre où il est essentiellement dirigé par la minimisation du CIP, j'ai regardé les valeurs de σ que l'algorithme a en améliorant la solution.

- 0.5, 0.5, 0.5, 0.5, 0.5, 0.012, 0.018, 0.091, 0.0137
- 0.5, 0.5, 0.5, 0.15, 0.04

L'évolution décroissante de σ est intéressante, elle n'est pas vraiment une conséquence de l'implémentation. Elle pourrait suggérer d'utiliser des paliers plus grands et d'augmenter le nombre itérations au total. Mais ce n'est pas c'est un aspect très secondaire.

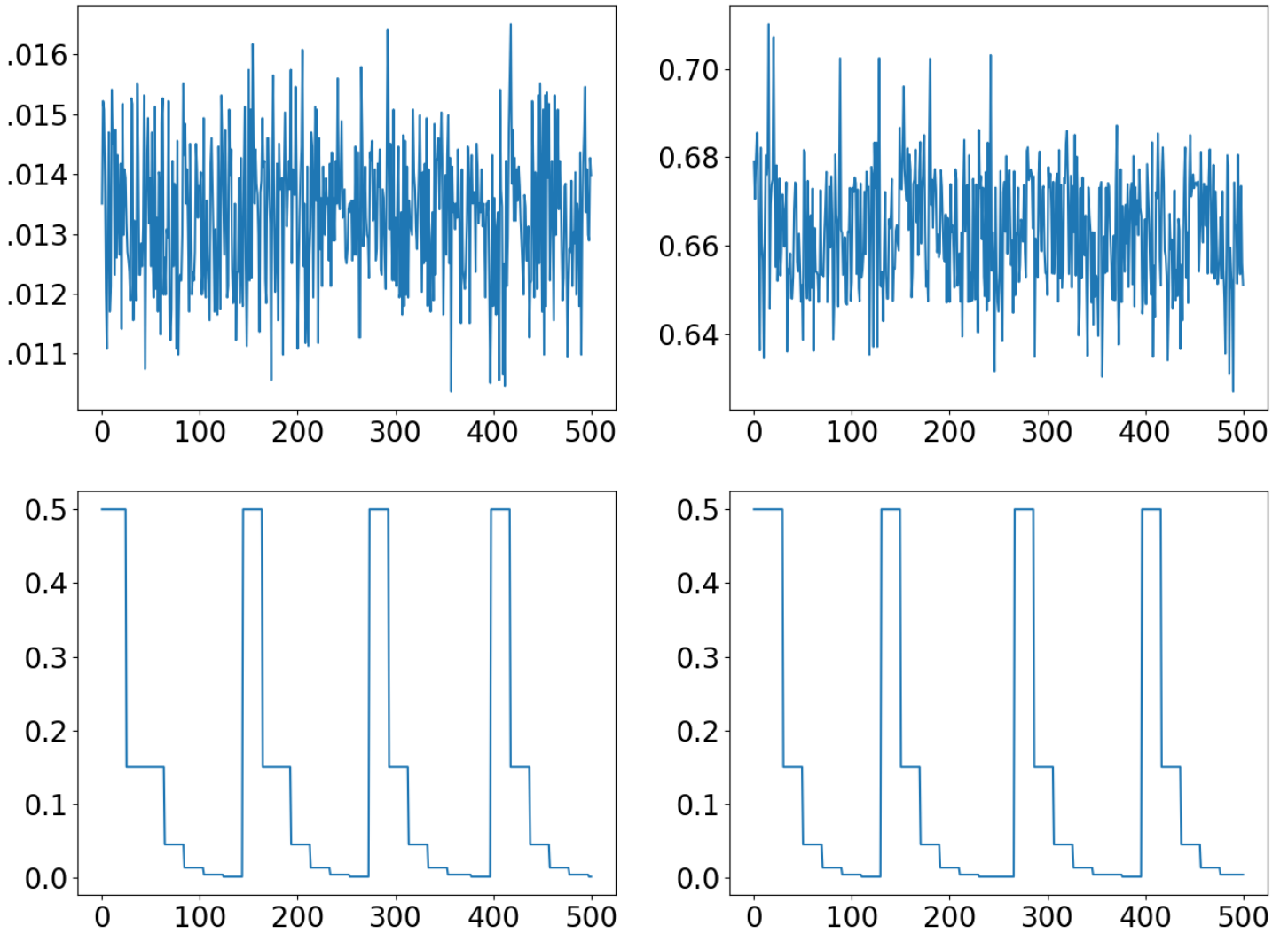


Figure 1: À gauche : évolution du CIP en fonction du nombre d'itérations. À droite : évolution du CE en fonction du nombre d'itérations. En bas : évolution de sigma correspondante.

La figure 1 montre l'évolution du CIP à gauche et du CE à droite. On peut dire qu' l'hypothèse sous-jacente de l'algorithme du recuit-simulé n'est pas vraiment respecter, même en étant très proche de la meilleure valeur, ce qui n'est pas du tout le cas même quand sigma est très faible. On sent pour la droite du graphique (pour CE), un plus grand lien entre sigma et les valeurs de CE.

Je pense que la raison est que la raison est le fait d'agir sur 200 longueurs paramètres en même temps. Je pense que c'est encore le *curse of dimensionality*. Effectivement je suis d'accord pour chercher une technique qui vise plus spécifiquement une ou plusieurs directions.

Il y aurait plein de façons de faire cela, l'objectif est de garder une implémentation pas trop complexe, d'utiliser les algorithmes existant et de chercher quand même à ce que l'idée soit prise en compte.

3 Première proposition

Curieusement en regardant la littérature sur les heuristiques similaires au recuit simulé, je n'ai pas trouvé ce que je cherche. Peut-être le *Adaptive Simulating Annealing* mais je n'ai pas trouvé la documentation claire.

L'intuition que je suis est d'apprendre pendant le déroulement de l'algorithme l'endroit où il faut chercher la meilleur solution.

Je propose de garder en mémoire non pas seulement le meilleur score mais aussi le vecteur qui a modifié la solution. Je propose ensuite de tirer aléatoirement une valeur réelle en utilisant sigma que je multiplie ensuite par ce vecteur qui a modifié la solution et que j'ai aussi normalisé. Je tire ensuite un vecteur en utilisant sigma. Enfin je considère la moyenne de ces deux vecteurs que j'ajoute au meilleur score.

De cette façon j'oriente la recherche dans la direction qui a le mieux réussi mais en en continuant quand même à explorer d'autres directions potentiellement prometteuses. On pourrait discuter d'une pondération, je fais la moyenne mais ne serait-il pas souhaitable de moins explorer en mettant une pondération moindre sur le deuxième terme. Plus précisément j'utilise les notations suivantes.

- t est le numéro de l'itération
- $\mathbf{x}_t$ le vecteur associé à l'itération t
- $\hat{\mathbf{x}}_t$ est le meilleur vecteur obtenu à l'itération t .
- $\mathbf{b}_t$ est vecteur tiré aléatoirement suivant une loi gaussienne centrée et d'écart-type 1.
- σ_t est la valeur de sigma à l'itération t .
- $f(\mathbf{x})$ est la valeur du score en fonction de $\mathbf{x}$.

L'algorithme tel qu'il fonctionne actuellement utilise les équations suivantes.

$$\begin{aligned}\mathbf{x}_{t+1} &= \hat{\mathbf{x}}_t + \sigma_t \mathbf{b}_t \\ \text{Si } f(\mathbf{x}_{t+1}) &> f(\hat{\mathbf{x}}_t) \text{ alors } \hat{\mathbf{x}}_{t+1} = \mathbf{x}_{t+1}\end{aligned}\tag{2}$$

- `noise` est désigné ici par $\sigma_t \mathbf{b}_t$.
- `best_score` est désigné ici par $f(\hat{\mathbf{x}}_t)$
- `best_weights` est désigné ici par $\hat{\mathbf{x}}_t$

Je rajoute maintenant les notations suivantes :

- $\widehat{\Delta \mathbf{x}}_t$ est la meilleur modification de vecteur à l'itération t .
- b'_t est la valeur tirée aléatoirement suivant une loi gaussienne centrée et d'écart-type 1.
- $\|\widehat{\Delta \mathbf{x}}_t\|$ est la norme Euclidienne de $\widehat{\Delta \mathbf{x}}_t$, de sorte que $\frac{\widehat{\Delta \mathbf{x}}_t}{\|\widehat{\Delta \mathbf{x}}_t\|}$ est de norme 1.

Ce que je propose maintenant.

$$\begin{aligned}\mathbf{x}_{t+1} &= \hat{\mathbf{x}}_t + \sigma_t \left(\frac{1}{2} b'_t \frac{\widehat{\Delta \mathbf{x}}_t}{\|\widehat{\Delta \mathbf{x}}_t\|} + \frac{1}{2} \mathbf{b}_t \right) \\ \text{Si } f(\mathbf{x}_{t+1}) &> f(\hat{\mathbf{x}}_t) \text{ alors } \hat{\mathbf{x}}_{t+1} = \mathbf{x}_{t+1} \text{ et } \widehat{\Delta \mathbf{x}}_{t+1} = \mathbf{x}_{t+1} - \hat{\mathbf{x}}_t\end{aligned}\tag{3}$$

4 Deuxième proposition

Une idée plus simple en terme d'implémentation est d'appliquer l'algorithme tel qu'il existe à la première longueur d'onde, de récupérer le poids idéal, puis de l'appliquer à une deuxième longueur d'onde et ainsi de suite. A chaque nouvelle longueur d'onde je ne m'autorise plus à modifier les poids déjà optimisé, je ne modifie que l'importance relative de ce que j'ai déjà optimisé avec ce que j'ajoute encore.

C'est naturellement sous-optimal, ne serait-ce que parce que cela dépend de l'ordre dans lequel j'introduis les nouvelles longueurs d'onde.

5 Troisième proposition

L'idée est différente. L'optimisation des fonctions à plusieurs variables est un sujet très étudié et il se trouve qu'il est implémenté dans les réseaux de neurones je propose donc d'utiliser ce savoir faire pour ce problème.

Je propose donc d'utiliser en entrée du réseau de neurones les 200 intensités d'un pixel quelconque. Je propose que la sortie du réseau de neurones est formé des 200 intensités du pixel déformés. Il importe que cette déformation soit la même pour tous les pixels. Et ici je me permet en plus d'envisager des déformations non-linéaires. En fait

elles seraient linéaires si j'imposais que le réseau de neurone soit à une couche. L'essentiel est que la fonction de coût à minimiser est le score avec un signe moins. Ce n'est pas l'utilisation classique des réseaux de neurones donc il y a un petit souci d'implémentation à vérifier. La fonction de coût n'est pas dérivable mais justement c'est un sujet très étudié, donc je crois comprendre qu'il y a des techniques qui s'en occupe. En pratique le réseau de neurone peut être utilisé sans carte graphique si le problème n'est pas trop complexe.

Pour vérifier que l'idée fonctionne bien, il est possible de ne considérer que quelques longueurs d'onde au lieu des 200.

6 Remarques sur le fichier `kmeans_vers3.py`

Je ne comprend pas pourquoi ce fichier n'est pas utilisable comme un module que j'aurai souhaité importé avec `import` et utilisé ses fonctions.

Du point informatique, des variables que j'appelle globales parce que définies en dehors des fonctions `data`, `gt`, `M,N,K`, `data_norm` ne sont pas utilisées en tant que variables globales, elles sont utilisées en tant que variables locales à travers l'appel. Je pense que c'est un choix risqué parce qu'il y a deux variables différentes qui portent le même nom et Python ne va pas vérifier qu'elles sont identiques. En pratique j'évite les variables globales, d'autant que Python accède à leurs contenu sans même que le mot clef `global` soit utilisé.

Dans le programme, `ce` est calculé deux fois tandis que `cip` n'est pas accessible pour un affichage ou un historique.